

XCS221 Assignment 6 — Bayesian Networks

Due Sunday, May 17 at 11:59pm PT.

Guidelines

1. If you have a question about this homework, we encourage you to post your question on our Slack channel, at <http://xcs221-scpd.slack.com/>
2. Familiarize yourself with the collaboration and honor code policy before starting work.
3. For the coding problems, you must use the packages specified in the provided environment description. Since the autograder uses this environment, we will not be able to grade any submissions which import unexpected libraries.

Submission Instructions

Written Submission: Some questions in this assignment require a written response. For these questions, you should submit a PDF with your solutions online in the online student portal. As long as the PDF is legible and organized, the course staff has no preference between a handwritten and a typeset $\text{\LaTeX}$ submission. If you wish to typeset your submission and are new to $\text{\LaTeX}$, you can get started with the following:

- Type responses only in `submission.tex`.
- Submit the compiled PDF, **not** `submission.tex`.
- Use the commented instructions within the `Makefile` and `README.md` to get started.

Also note that your answers should be in order and clearly and correctly labeled to receive credit. Be sure to submit your final answers as a PDF and **tag all pages correctly when submitting to Gradescope**.

Coding Submission: Some questions in this assignment require a coding response. For these questions, you should submit only the `src/submission.py` file in the online student portal. For further details, see Writing Code and Running the Autograder below.

Honor code

We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down the solutions independently, and without referring to written notes from the joint session. In other words, each student must understand the solution well enough in order to reconstruct it by him/herself. In addition, each student should write on the problem set the set of people with whom s/he collaborated. Further, because we occasionally reuse problem set questions from previous years, we expect students not to copy, refer to, or look at the solutions in preparing their answers. It is an honor code violation to intentionally refer to a previous year's solutions. More information regarding the Stanford honor code can be found at <https://communitystandards.stanford.edu/policies-and-guidance/honor-code>.

Writing Code and Running the Autograder

All your code should be entered into `src/submission.py`. When editing `src/submission.py`, please only make changes between the lines containing `### START_CODE_HERE ###` and `### END_CODE_HERE ###`. Do not make changes to files other than `src/submission.py`.

The unit tests in `src/grader.py` (the autograder) will be used to verify a correct submission. Run the autograder locally using the following terminal command within the `src/` subdirectory:

```
$ python grader.py
```

There are two types of unit tests used by the autograder:

- **basic:** These tests are provided to make sure that your inputs and outputs are on the right track, and that the hidden evaluation tests will be able to execute.
- **hidden:** These unit tests are the evaluated elements of the assignment, and run your code with more complex inputs and corner cases. Just because your code passed the basic local tests does not necessarily mean that they will pass all of the hidden tests. These evaluative hidden tests will be run when you submit your code to the Gradescope autograder via the online student portal, and will provide feedback on how many points you have earned.

For debugging purposes, you can run a single unit test locally. For example, you can run the test case `3a-0-basic` using the following terminal command within the `src/` subdirectory:

```
$ python grader.py 3a-0-basic
```

Before beginning this course, please walk through the [Anaconda Setup for XCS Courses](#) to familiarize yourself with the coding environment. Use the env defined in `src/environment.yml` to run your code. This is the same environment used by the online autograder.

Test Cases

The autograder is a thin wrapper over the python `unittest` framework. It can be run either locally (on your computer) or remotely (on SCPD servers). The following description demonstrates what test results will look like for both local and remote execution. For the sake of example, we will consider two generic tests: `1a-0-basic` and `1a-1-hidden`.

Local Execution - Hidden Tests

All hidden tests rely on files that are not provided to students. Therefore, the tests can only be run remotely. When a hidden test like `1a-1-hidden` is executed locally, it will produce the following result:

```
----- START 1a-1-hidden: Test multiple instances of the same word in a sentence.
----- END 1a-1-hidden [took 0:00:00.011989 (max allowed 1 seconds), ???/3 points] (hidden test ungraded)
```

Local Execution - Basic Tests

When a basic test like `1a-0-basic` passes locally, the autograder will indicate success:

```
----- START 1a-0-basic: Basic test case.
----- END 1a-0-basic [took 0:00:00.000062 (max allowed 1 seconds), 2/2 points]
```

When a basic test like `1a-0-basic` fails locally, the error is printed to the terminal, along with a stack trace indicating where the error occurred:

```
----- START 1a-0-basic: Basic test case.
<class 'AssertionError'>
{'a': 2, 'b': 1} != None ← This error caused the test to fail.
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a") ← In this case, start your debugging
                                           in line 23 of grader.py.
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
----- END 1a-0-basic [took 0:00:00.003809 (max allowed 1 seconds), 0/2 points]
```

Remote Execution

Basic and hidden tests are treated the same by the remote autograder. Here are screenshots of failed basic and hidden tests. Notice that the same information (error and stack trace) is provided as the in local autograder, now for both basic and hidden tests.

1a-0-basic) Basic test case. (0.0/2.0)

```
<class 'AssertionError': {'a': 2, 'b': 1} != None
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a"))
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

Just like in the local autograder, this error caused the test to fail.

Just like in the local autograder, start your debugging in line 23 of grader.py.

1a-1-hidden) Test multiple instances of the same word in a sentence. (0.0/3.0)

```
<class 'AssertionError': {'a': 23, 'ab': 22, 'aa': 24, 'c': 16, 'b': 15} != None
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/grader.py", line 31, in test_1
    self.compare_with_solution_or_wait(submission, 'extractWordFeatures', lambda f: f(sentence))
File "/autograder/source/graderUtil.py", line 183, in compare_with_solution_or_wait
    self.assertEqual(ans1, ans2)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

This error caused the test to fail.

Start your debugging in line 31 of grader.py.

Finally, here is what it looks like when basic and hidden tests pass in the remote autograder.

1a-0-basic) Basic test case. (2.0/2.0)

1a-1-hidden) Test multiple instances of the same word in a sentence. (3.0/3.0)

1. Basic Bayes

Let's consider a simple scenario which we can model using a Bayesian network. You are walking to CS221 lecture and, on the way, you notice the grass on the Stanford Oval is wet. You reason that this could be because either it rained (rare in beautiful and sunny Palo Alto) or the sprinklers were on. Letting your mind wander from the lecture, you decide to sit down and figure out what happened here by applying your knowledge of Bayesian networks.

- (a) [1 point (Written)] **Bayesian Network Diagram** List the edges in the Bayesian network that is a causal model for why the grass on the Oval is wet. Use the variables Rain, Sprinkler, and WetGrass.

What we expect: A list of edges in the Bayesian network, in the format of variable1 $\rightarrow$ variable2.

- (b) [1 point (Written)] **Joint Probability Expression**

Express the joint probability $\mathbb{P}(\text{Rain}, \text{Sprinkler}, \text{WetGrass})$ in terms of local conditional probabilities.

What we expect: The simplified joint probability expression, in terms of the local conditional probabilities (not a numerical value).

- (c) [1 point (Written)] **Marginal Probability**

Express $\mathbb{P}(\text{WetGrass})$ in terms of local conditional probabilities.

What we expect: The simplified marginal probability expression, in terms of the local conditional probabilities (not a numerical value).

- (d) [1 point (Written)] **Likelihood Calculation**

Assume $p(\text{Rain}) = 0.2$, $p(\text{Sprinkler}) = 0.5$, and the following local conditional probability table for $p(\text{WetGrass} \mid \text{Rain}, \text{Sprinkler})$:

Rain	Sprinkler	$p(\text{WetGrass} \mid \text{Rain}, \text{Sprinkler})$
True	True	0.9
True	False	0.8
False	True	0.7
False	False	0.0

What is the probability that it rained, the sprinklers are on, but the grass is not wet?

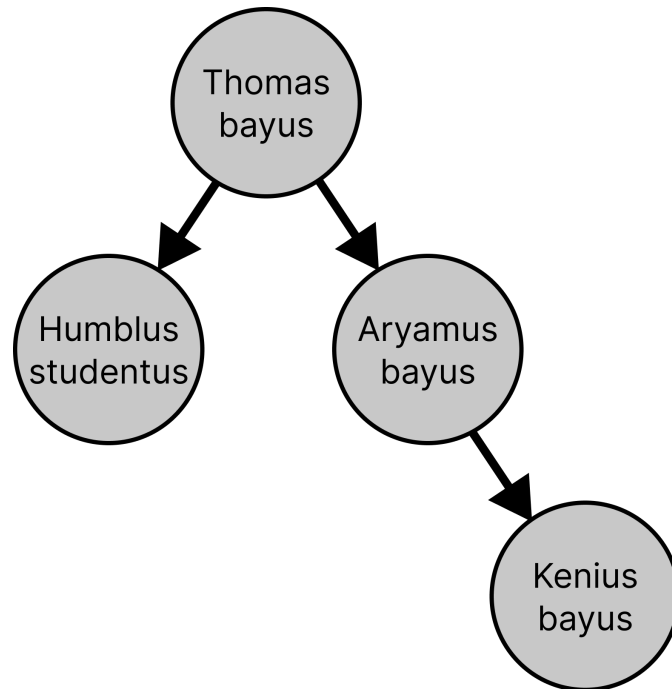
What we expect: A numerical answer with brief calculations shown.

- (e) [1 point (Written)] **Independence Analysis**

- (a) Rain and Sprinkler are independent.
- (b) Rain and Sprinkler are conditionally independent given WetGrass.
- (c) $\mathbb{P}(\text{Rain} \mid \text{WetGrass}) > \mathbb{P}(\text{Rain} \mid \text{WetGrass}, \text{Sprinkler})$.

What we expect: T/F for each subpart. No justification needed.

2. Sampling



You got distracted while examining that luscious green grass in the Stanford Oval, and you somehow wandered into the Gilbert Biological Sciences Building. As you realize where you are, you overhear some biologists discussing the genome of a new species they've discovered, provisionally given the scientific name *Kenius bayus*.

(As you remember from high school biology, the genome of a living thing is encoded in DNA, which is a long chain of nucleotides. Each nucleotide can be one of four organic molecules: *A*, *C*, *T*, or *G*.)

Assume a simplified model of biology where each descendant mutates the gene of its ancestor with a probability of `mutation_rate`, with a uniform distribution over the other three nucleotides when mutating. The mutation process is independent for each nucleotide.

For example, if we set `mutation_rate = 0.1`, then the local conditional probability table for $p(\text{HumblusStudentus} \mid \text{ThomasBayus})$ would be:

ThomasBayus	$p(\text{HS} = \text{A} \mid \text{TB})$	$p(\text{HS} = \text{C} \mid \text{TB})$	$p(\text{HS} = \text{T} \mid \text{TB})$	$p(\text{HS} = \text{G} \mid \text{TB})$
A	0.9	0.03333	0.03333	0.03333
C	0.03333	0.9	0.03333	0.03333
T	0.03333	0.03333	0.9	0.03333
G	0.03333	0.03333	0.03333	0.9

The biologists are trying to figure out the probability of *Kenius bayus*'s genome resulting from a series of mutations from its ancestor, *Thomas bayus*. You realise you could help these poor biologists, who never took CS 221, by modelling the mutation process with Bayesian networks!

- (a) **[2 points (Coding)]** Let's first get familiar with the provided interface for Bayesian networks. Convert the provided phylogenetic tree into a Bayesian network using the `BayesianNetwork` class and the `BayesianNode` class defined in `util.py`.

Understanding the shape of the conditional probability table (CPT): When you create a `BayesianNode`, the last dimension of its conditional probability table will correspond to that node's domain, and every preceding dimension will match the size of each parent's domain (in parent order). For

example, a node with two parents whose domains are length 4 and a child domain of length 4 needs a CPT of shape (4, 4, 4).

By default, the `BayesianNode` constructor will create a uniform distribution over the domain, but for some problems you may need to set the CPTs to a different distribution.

For later problems: Remember to read and understand the member functions of the `BayesianNode` class in `util.py`, particularly `get_probability`, `parent_assignment_indices`, and `iter_parent_assignments`.

Also, each `BayesianNetwork` has a `batch_size` parameter, which sets the size of a special batch axis for the nodes without parents. Your code may have to handle such nodes as a special case. This batch axis is used to sample sequences of independent observations (e.g. perhaps a DNA sequence...). By default, the `batch_size` is 1, but for some problems you may need to set it to a different value.

What we expect: An implementation of the `initialize_phylogenetic_tree` function in `submission.py`, which takes in a `mutation_rate` and returns a `BayesianNetwork` object representing the phylogenetic tree with the conditional probability tables being determined by the mutation rate.

We provide code for varying the `genome_length` parameter; you only need to define each node and its domain, parents, and conditional probability table.

(b) [2 points (Coding)]

You will need to implement some initial scaffolding to enable sampling from Bayesian networks in the code. Extend the `BayesianNetwork` class and implement the function `forward_sampling`. This function should sample a single observation from the joint probability distribution of the given Bayesian network.

What we expect: An implementation of the `forward_sampling` function in `submission.py`. You should use the given ordering of variables in `self.order` and sample from each one in order, using its conditional probability distribution and the values of its parents (which you should have already sampled). Each variable is represented by a `BayesianNode` object. Your solution should explicitly handle varying sequence lengths by using the `network.batch_size` parameter.

(c) [2 points (Coding)]

Implement `compute_joint_probability`, which computes the joint probability of a given set of assignments to the variables in the Bayesian network.

What we expect: An implementation of the `compute_joint_probability` function in `submission.py`. Your solution should explicitly handle varying sequence lengths by using the `network.batch_size` parameter.

(d) [1 point (Written)] **Forward Sampling Table**

Implement `test_forward_sampling`, which will initialize the network with `mutation_rate = 0.1` and `genome_length = 10`, and then run the `forward_sampling` function once on this network. Fill out the table below with the complete genomes you observed for each species.

Species	Genome
Thomas bayus	
Humblus studentus	
Aryamus bayus	
Kenius bayus	

What we expect: A completed table showing the sampled values for each variable in the phylogenetic tree.

(e) [3 points (Coding)]

Now we will use **rejection sampling** to estimate the following posterior probability: $\mathbb{P}(\text{ThomasBayus} = \text{AAAC} \mid \text{KeniusBayus} = \text{AAAA})$.

Implement the function `rejection_sampling`, which takes in a setting for *Kenius bayus*'s gene (as a list of nucleotide strings, e.g. ['A', 'A', 'A', 'A']) along with the number of samples as an argument.

What we expect: An implementation of the `rejection_sampling` function in `submission.py`. Your implementation should directly call the `forward_sampling` function you implemented in part (b). As before, your solution should explicitly handle the sequence length dimension by using the `network.batch_size` parameter.

(f) [3 points (Coding)]

We will now implement **Gibbs sampling** because these biologists are getting impatient. Complete the implementation of the iteration loop in the `gibbs_sampling` function, which takes in a dictionary with settings for the evidence variables along with number of iterations. We have implemented the initial assignment for you.

What we expect: An implementation of the iteration loop in the `gibbs_sampling` function in `submission.py`.

(g) [1 point (Written)]

Fill out the table below with the estimated posterior probabilities from each method.

Method	Steps/Iterations	Estimated Posterior
Rejection sampling	100	
Rejection sampling	10,000	
Gibbs sampling	100	
Gibbs sampling	10,000	
Exact	—	

What we expect: A completed table comparing rejection sampling, Gibbs sampling, and exact inference results.

3. Learning

Later in life, you're a data engineer managing a team of human annotators. Your data consists of labels indicating whether a given conversation between an LLM and a human was **good** or **bad**.

But you've run into a problem: your annotators can't seem to perfectly agree on their ratings, no matter how careful you are in your instructions! You vaguely remember something about Bayesian networks from CS221, so you decide to use them to help you assess annotator quality.

For this problem, we'll assume you have ground-truth labels for each data point. (In the real world, this will not be the case!)

(a) [1 point (Written)] **Annotator Network Diagram**

Let's assume you have a single data point and three annotators who assign labels to this data point. List the edges in the Bayesian network for this scenario using the random variables Y (the prior over ground-truth labels of this data point), A_1 , A_2 , and A_3 (the distribution over the annotators' claimed labels conditioned on the ground-truth label of the data point).

What we expect: A list of edges in the Bayesian network, in the format of `variable1 → variable2`.

(b) [1 point (Written)] **Perfect Annotator CPT**

Fill out the following table with the local conditional probability distribution of a perfect annotator.

Y	$p(A_1 = \text{good} \mid Y)$	$p(A_1 = \text{bad} \mid Y)$
good		
bad		

What we expect: A table showing the conditional probability distribution.

(c) [1 point (Written)] **Random Annotator CPT**

Fill out the following table with the local conditional probability distribution of an annotator who guesses uniformly at random.

Y	$p(A_1 = \text{good} \mid Y)$	$p(A_1 = \text{bad} \mid Y)$
good		
bad		

What we expect: A table showing the conditional probability distribution.

(d) [4 points (Coding)]

Implement `bayesian_network_for_annotators(num_annotators, dataset_size)`, which returns the Bayesian network used for the remaining parts of this problem set. Your network should contain a latent Y node for the labels and one annotator node per annotator (e.g. A_0 , A_1 , A_2 ; 0-indexed), all sharing the same domain `['good', 'bad']`.

What we expect: Describe the nodes, domains, parent relationships, and initial CPTs you use. The CPTs should encode a reasonable prior that annotators are better than chance (e.g., diagonally dominant matrices).

(e) [3 points (Coding)]

We will now implement maximum-likelihood estimation for Bayesian network parameters, which assumes every variable is observed. Implement the function `mle_estimate`, which takes in a `BayesianNetwork` and a list of observations and returns a new `BayesianNetwork` with the parameters estimated by MLE.

You will have to support Laplace smoothing using the parameter `lambda_param`.

What we expect: An implementation of the `mle_estimate` function in `submission.py`.

You should first implement the helper function `accumulate_assignment` to accumulate the counts for each assignment. You should also use the provided helper functions `init_zero_conditional_probability_tables` and `normalize_counts` where applicable; you don't need to implement them.

(f) [3 points (Coding)]

We have provided a training dataset `annotations.csv` with data from 3 annotators on 100 data points. Note that this dataset contains ground-truth labels for each data point. Read in the dataset, initialise the correct `BayesianNetwork` structure for modelling this problem, and use `mle_estimate` to learn its parameters.

What we expect: Implement `mle_estimate_for_annotators`, which takes in a list of observations and returns a new `BayesianNetwork` object with the parameters estimated by MLE.

(g) [1 point (Written)] **Annotator Analysis**

After training the model with MLE, plot the conditional probability table for each of the annotators using the function `plot_annotator_cpts`. Which annotator seems to be the least trustworthy, and why?

What we expect: A plot of the conditional probability tables for each annotator, along with a brief explanation of why the annotator is the least trustworthy.

4. Learning without Labels

Your real dataset doesn't actually have ground-truth labels. You thus want to both model the annotator noise as well as estimate the ground-truth labels.

You must now use the **Expectation Maximization (EM)** algorithm. Implement the algorithm and analyze the results on the dataset.

(a) [6 points (Coding)]

Implement `e_step(BayesianNetwork, data)` which creates a dataset of fully-observed observations along with their normalized weights $q(h)$. (You should be able to use functions you implemented earlier as subroutines.)

What we expect: An implementation of the `e_step` function in `submission.py`. We expect you to use the helper functions we've provided as well as your own implementations of `compute_joint_probability` and `accumulate_assignment`.

Tip: you should normalize the weights for each sample in the dataset independently, e.g. the weights for all possible labels for `sample0` should sum to 1.

(b) [4 points (Coding)]

Implement `m_step(BayesianNetwork, data)` which takes in the dataset from the `e_step` function and updates the parameters of the Bayesian network using maximum likelihood estimation.

What we expect: An implementation of the `m_step` function in `submission.py`. Your code should use your earlier implementations of `accumulate_assignment` and the provided `init_zero_conditional_probability_tables` and `normalize_counts`.

(c) [2 points (Coding)]

Implement `em_learn(BayesianNetwork, data, num_iterations)` which runs the EM algorithm for a given number of iterations.

What we expect: An implementation of the `em_learn` function in `submission.py`.

(d) [1 point (Written)] **EM Plots**

Use the provided function `test_em_learn` to train the model for 100 iterations. Plot the conditional probability tables for each of the annotators and the labels using the functions `plot_annotator_cpts` and `plot_label_cpt`. Show both resulting plots.

What we expect: Two plots of the conditional probability tables for each of the annotators and the labels.

(e) [1 point (Written)] **EM Recovery Analysis**

Based on the plots, did the EM algorithm recover the true labels? How do you know?

What we expect: A yes/no answer to the question, along with the evidence for your answer.

5. Product Ethics Exploration: Economics

You will choose a modern AI product to analyze throughout the assignments and gain initial access to it.

Suggested resources:

- Pick an AI-powered product or tool (not just a raw LLM chatbot) that is accessible to you. Good candidates include: Cursor, Nano Banana, ElevenLabs, Perplexity, Claude Code, etc. Free trials or student-accessible tools are recommended.
- The product's official website and onboarding/getting started pages
- Privacy policy, Terms of Service, Acceptable Use Policy
- App store listings, GitHub documentation or READMEs (if open-source or developer-targeted)
- Recent news coverage (e.g., TechCrunch, The Verge, Wired, NYT technology section)
- Reddit communities, Hacker News, or user forums for first impressions and experiences

In this part: You will examine the economic and labor impacts of the product, including job displacement, augmentation, and effects on societal inequality.

Why it's important: AI systems can reshape labor markets and either reduce or exacerbate existing economic disparities.

Suggested resources:

- Industry reports (e.g., McKinsey, PwC, U.S. Bureau of Labor Statistics) on job displacement trends
- Blog posts by the company about use cases or productivity gains
- Academic articles analyzing the impacts of automation
- Tech policy think tanks (Brookings, AI Now Institute, etc.)

(a) [0.50 points (Written)] Job Tasks and Potential for Displacement

- Identify and describe what kinds of tasks or jobs this product is most useful for.
- Refer to at least one resource (industry report, company blog, or academic article) that discusses these types of tasks or jobs, and summarize any projections or claims about automation or productivity within this context.
- Consider the **ecosystem view**: AI's impact depends on upstream factors (data, compute, labor, environment) and downstream factors (users, deployment, harm). Beyond job displacement, what are the upstream labor implications? What kinds of human labor (e.g., data labeling, content moderation, model training) were required to build and maintain this product?

What we expect: In 4-6 sentences, list and discuss the primary tasks or job roles relevant to this product. Support your answer with at least one citation to an external source when possible. Additionally, discuss the upstream labor required to build and maintain the product, considering the full ecosystem of work involved.

(b) [0.50 points (Written)] Job Displacement vs. Augmentation

- Discuss whether this product is more likely to directly replace existing jobs or to augment (assist/support) workers in those jobs. Explain your reasoning based on product features and cited resources, if possible.
- Analyze the potential consequences and risks if the product replaces or augments jobs incorrectly or incompetently. What could go wrong for both workers and employers?

What we expect: In 4-6 sentences, provide a thoughtful analysis of job displacement versus augmentation, including at least one risk or downside for workers. Support your conclusions with observations about the product or external evidence.

(c) [0.50 points (Written)] Societal Inequalities

- Analyze how the adoption of this tool might either exacerbate or mitigate existing societal inequalities (e.g., digital divide, socioeconomic, class, racial, or geographic divides).
- Consider which demographics of users benefit most versus least from this technology. Would increased access to the product help close such divides, or does it risk deepening them?

- Building on the concept of **inequality** in AI systems: beyond access, consider how the product’s performance might vary across different user groups. How might differences in performance (not just access) contribute to inequality? What metrics should be monitored to ensure equitable outcomes?

What we expect: Identify at least one way the product could impact societal inequalities, and discuss specific groups of users who would be most and least likely to benefit. Additionally, in 2-3 sentences, analyze how performance disparities (not just access barriers) might contribute to inequality, and suggest metrics for monitoring equitable outcomes.

(d) [0.50 points (Written)] **Strategies and Mitigation Measures**

- Propose at least one concrete strategy for deploying or regulating this technology to lessen negative effects (such as job loss or deepening inequality). Possibilities include “human-in-the-loop” mechanisms, reskilling or retraining programs, oversight requirements, or policy interventions.
- Discuss how much power or oversight humans should have over the system’s predictions and decisions in these mitigation approaches.

What we expect: In 4-6 sentences, describe a pragmatic mitigation strategy and briefly explain how it would help reduce negative economic or social outcomes. Address the role of human oversight or intervention as part of the solution.

6. Product Ethics Exploration: Rebuilding

In this part: You will synthesize your findings to design an improved version of the product and reflect on who should have power and responsibility in building AI systems.

Why it's important: This synthesis helps you think critically about how to build better AI systems and who should be accountable for their impacts, shifting from theory to implementation.

Suggested Resources:

- Responsible AI guidelines ([OECD](#), [NIST AI Risk Management Framework](#))
- Other tools in the same product category for comparison
- Human-Computer Interaction (HCI) or User Experience (UX) design guidelines
- Ethical AI frameworks discussed in class and previous assignments

(a) [0.50 points (Written)] Summarize and Improve Your Product

- Based on the research you've done in this homework, summarize your chosen product's key features, strengths, and weaknesses in 1 paragraph.
- Write a 1-2 paragraph summary of actionable improvements you would make to the product to address the weaknesses you identified.

What we expect: In 2-3 paragraphs, provide a thoughtful summary of your research on your chosen product and actionable improvements you would make to the product to address the weaknesses you identified.

(b) [0.50 points (Written)] Power and Responsibility in Building AI

- Who currently has the power to make meaningful changes to products like this: engineers, designers, executives, policymakers, users, or someone else? Who do you think *should* hold the power and responsibility for building better AI-powered tools, and why?
- Reflect on any tensions between who currently drives change and who ought to be responsible from a societal perspective. Propose one recommendation for shifting power or responsibility if there's a misalignment.

What we expect: In 4-6 sentences, discuss who builds and steers AI products like the one you selected, and who should be responsible for their positive and negative impacts, supporting your recommendations with evidence or reasoning.

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset L^AT_EX solutions.

- 1.a
- 1.b
- 1.c
- 1.d
- 1.e

2.d

2.g

3.a

3.b

3.c

3.g

4.d

4.e

5.a
5.b
5.c
5.d

6.a

6.b